

AI Assistant Coding

Lab - 6.1

Question 1:

Task Description #1 (AI-Based Code Completion for Loops)

Task: Use an AI code completion tool to generate a loop-based program.

Prompt:

“Generate Python code to print all even numbers between 1 and N using a loop.”

Expected Output:

- AI-generated loop logic.
- Identification of loop type used (for or while).
- Validation with sample inputs

Prompt :

```
#generate a code to print all even numbers between 1 to n using a loop
```

Code :

```
#generate a code to print all even numbers between 1 to n using a loop
def print_even_numbers(n):
    for num in range(1, n + 1):
        if num % 2 == 0:
            print(num, end=' ')
#input n
n = int(input("Enter a number n: "))
print(f"Even numbers between 1 and {n}:")
print_even_numbers(n)
```

Output :

```
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\lab4.py"
Enter a number n: 50
Even numbers between 1 and 50:
2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50
❖ PS C:\Users\dasar\AI-Assistant-Coding> 
```

Analysis :

The code correctly uses a loop and conditional statement to print even numbers between 1 and `n`. It works for valid positive integers but does not handle invalid or negative inputs.

Question 2:

Task Description #2 (AI-Based Code Completion for Loop with Conditionals)

Task: Use an AI code completion tool to combine loops and conditionals.

Prompt:

“Generate Python code to count how many numbers in a list are even and odd.”

Expected Output:

- AI-generated code using loop and if condition.
- Correct count validation.
- Explanation of logic flow

Prompt :

```
#Generate Python code to count how many numbers in a list are even and odd
```

Code :

```
#Generate Python code to count how many numbers in a list are even and odd
def count_even_odd(numbers):
    even_count = 0
    odd_count = 0
    for num in numbers:
        if num % 2 == 0:
            even_count += 1
        else:
            odd_count += 1
    return even_count, odd_count
#example list
numbers = list(map(int, input("Enter numbers separated by spaces: ").split()))
even_count, odd_count = count_even_odd(numbers)
print(f"Even numbers count: {even_count}")
print(f"Odd numbers count: {odd_count}")
```

Output :

```
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\lab4.py"
Enter numbers separated by spaces: 1 2 3 4 5 6 7 8 9 10
Even numbers count: 5
Odd numbers count: 5
PS C:\Users\dasar\AI-Assistant-Coding> █
```

Analysis :

The program accurately counts even and odd numbers using a loop with conditional checks. It produces correct results for integer inputs but assumes valid numeric input.

Question 3:

Task Description #3 (AI-Based Code Completion for Class Attributes Validation)

Task: Use an AI tool to complete a Python class that validates user input.

Prompt:

“Generate a Python class User that validates age and email using conditional statements.”

Expected Output:

- AI-generated class with validation logic.
- Verification of condition handling.
- Test cases for valid and invalid inputs

Prompt :

```
#Generate a Python class User that validates age and email using conditional statements
```

Code :

```
#Generate a Python class User that validates age and email using conditional statements
class User:
    def __init__(self, name, age, email):
        self.name = name
        self.age = age
        self.email = email

    def is_valid_age(self):
        return 0 < self.age < 120

    def is_valid_email(self):
        return "@" in self.email and "." in self.email.split("@")[-1]
```

```

def validate_user(self):
    age_valid = self.is_valid_age()
    email_valid = self.is_valid_email()
    return age_valid, email_valid
#example usage
name = input("Enter your name: ")
age = int(input("Enter your age: "))
email = input("Enter your email: ")
user = User(name, age, email)
age_valid, email_valid = user.validate_user()
print(f"Age valid: {age_valid}")
print(f"Email valid: {email_valid}")

```

Output :

```

PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\lab4.py"
Enter your name: siri
Enter your age: 20
Enter your email: siri@email.com
Age valid: True
Email valid: True
PS C:\Users\dasar\AI-Assistant-Coding> 

```

Analysis :

The `User` class correctly validates age and email using conditional logic. Email validation is basic and does not cover all possible invalid formats.

Question 4:

Task Description #4 (AI-Based Code Completion for Classes)

Task: Use an AI code completion tool to generate a Python class for managing student details.

Prompt:

“Generate a Python class Student with attributes (name, roll number, marks) and methods to calculate total and average marks.”

Expected Output:

- AI-generated class code.
- Verification of correctness and completeness of class structure.
- Minor manual improvements (if needed) with justification

Prompt :

```
#Generate a Python class Student with attributes (name, roll number,marks) and methods to calculate total and average marks
```

Code :

```
#Generate a Python class Student with attributes (name, roll number,marks) and methods to calculate total and average marks
class Student:
    def __init__(self, name, roll_number, marks):
        self.name = name
        self.roll_number = roll_number
        self.marks = marks # marks should be a list of integers

    def total_marks(self):
        return sum(self.marks)

    def average_marks(self):
        if len(self.marks) == 0:
            return 0
        return self.total_marks() / len(self.marks)

#example usage
name = input("Enter student name: ")
roll_number = input("Enter roll number: ")
marks = list(map(int, input("Enter marks separated by spaces: ").split()))
student = Student(name, roll_number, marks)
total = student.total_marks()
average = student.average_marks()
print(f"Total marks for {student.name} (Roll No: {student.roll_number}): {total}")
print(f"Average marks: {average}")
```

Output :

```
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\lab4.py"
Enter student name: siri
Enter roll number: 1236
Enter marks separated by spaces: 90 89 99 87 73 77
Total marks for siri (Roll No: 1236): 515
Average marks: 85.83333333333333
PS C:\Users\dasar\AI-Assistant-Coding> 
```

Analysis :

The `Student` class correctly calculates total and average marks using class methods. It handles empty marks safely but lacks input range validation.

Question 5:

Task Description 5 (AI-Assisted Code Completion Review)

Task: Use an AI tool to generate a complete Python program using classes, loops, and conditionals together.

Prompt:

"Generate a Python program for a simple bank account system using class, loops, and conditional statements."

Expected Output:

- Complete AI-generated program.
- Identification of strengths and limitations of AI suggestions.
- Reflection on how AI assisted coding productivity

Prompt :

```
#Generate a Python program for a simple bank account system using
class, loops, and conditional statements
```

Code :

```
#Generate a Python program for a simple bank account system using
class, loops, and conditional statements
class BankAccount:
    def __init__(self, account_holder, balance=0):
        self.account_holder = account_holder
        self.balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"Deposited: ${amount}. New balance:
${self.balance}")
        else:
            print("Deposit amount must be positive.")

    def withdraw(self, amount):
        if amount > 0:
            if amount <= self.balance:
                self.balance -= amount
```

```

        print(f"Withdrew: ${amount}. New balance:
${self.balance}")
    else:
        print("Insufficient funds.")
    else:
        print("Withdrawal amount must be positive.")

    def get_balance(self):
        return self.balance
# Example usage
account_holder = input("Enter account holder name: ")
account = BankAccount(account_holder)
while True:
    print("\nOptions: 1. Deposit 2. Withdraw 3. Check Balance 4. Exit")
    choice = input("Choose an option (1-4): ")
    if choice == '1':
        amount = float(input("Enter amount to deposit: "))
        account.deposit(amount)
    elif choice == '2':
        amount = float(input("Enter amount to withdraw: "))
        account.withdraw(amount)
    elif choice == '3':
        balance = account.get_balance()
        print(f"Current balance: ${balance}")
    elif choice == '4':
        print("Exiting the program.")
        break
    else:
        print("Invalid option. Please try again.")

```

Output :

```
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\lab4.py"
Enter account holder name: siri

Options: 1. Deposit 2. Withdraw 3. Check Balance 4. Exit
Choose an option (1-4): 2
Enter amount to withdraw: 1500
Insufficient funds.

Options: 1. Deposit 2. Withdraw 3. Check Balance 4. Exit
Choose an option (1-4): 1
Enter amount to deposit: 2000
Deposited: $2000.0. New balance: $2000.0

Options: 1. Deposit 2. Withdraw 3. Check Balance 4. Exit
Choose an option (1-4): 2
Enter amount to withdraw: 1500
Withdrew: $1500.0. New balance: $500.0

Options: 1. Deposit 2. Withdraw 3. Check Balance 4. Exit
Choose an option (1-4): 3
Current balance: $500.0

Options: 1. Deposit 2. Withdraw 3. Check Balance 4. Exit
Choose an option (1-4): 4
Exiting the program.
PS C:\Users\dasar\AI-Assistant-Coding> █
```

Analysis :

The bank account program correctly integrates classes, loops, and conditionals for user interaction. It functions as expected but does not handle invalid numeric input or data persistence.